

agent: discovery-security-agent llm: claude-opus-4-6 run_id: 20260930T121134_g4zvb3 generated_at: 2026-09-30T06:56:04.000Z

6. Security Hotspots Analysis

Objective: Address key OWASP-class security vulnerabilities and dependency risk.

Date: 2026-09-30 06:56:04 UTC | **Scope:** pingcrm — Laravel 11 (PHP ^8.2), React 19 (Inertia.js), SQLite, Sanctum, Vite

Executive Summary

EXECUTIVE SUMMARY

The pingcrm codebase presents a **High Risk** security posture driven primarily by pervasive SQL injection vulnerabilities and unsafe `extract()` calls across the legacy IVR module layer. Eighty IVR controllers concatenate user-supplied query parameters directly into raw SQL strings without parameterization, creating exploitable injection vectors. The same 80 controllers call `extract($payload)` on unsanitized request data, enabling variable overwrite attacks. Twelve legacy "GodService" classes contain hardcoded API keys committed to source. On the frontend, the React pagination component uses `dangerouslySetInnerHTML` to render pagination labels, and the login page ships with pre-filled demo credentials. The CRM controllers (Users, Contacts, Organizations) follow Laravel best practices with Eloquent ORM and proper validation, but lack any authorization policy — any authenticated user can modify any other user's data within the same account. No security headers (CSP, HSTS, X-Frame-Options) are configured. No SAST, dependency scanning, or secret detection is present in CI. Both backend and frontend layers were reviewed.

91

FILES SCANNED FOR INPUT HANDLING

8

CONCRETE INJECTION/XSS/CSRF/CORS/AUTH FINDINGS

2

DEPENDENCIES FLAGGED OUTDATED/VULNERABLE

7/10

OWASP CATEGORIES WITH FINDINGS

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by 80 controllers with unparameterized SQL injection, 80 controllers with unsafe `extract()` on request data, and 12 hardcoded API keys in source.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	2	HIGH RISK
H2	High Vulnerabilities	0	<5	5–10	>10	4	GOOD
H3	Medium Vulnerabilities	low	<20	20–50	>50	6	GOOD
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	0.07/KLOC	GOOD
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	25% clean (3/12)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	2	HIGH RISK
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~12%	MODERATE
H8	End-of-Life Dependencies	0	0	1–5	>5	1	MODERATE

6.2 Hotspot-by-Hotspot Evidence

SQL Injection via String Concatenation in IVR Controllers **CRITICAL**

Eighty IVR controllers under `app/Http/Controllers/Ivr/` concatenate the user-supplied `q` request parameter directly into raw SQL strings passed to `DB::select()`. The `$q` value comes from `$request->get("q")` with no sanitization, escaping, or parameterization.

Example 1 — `app/Http/Controllers/Ivr/AgentDeskStoreController.php:28` :

```
$q = $request->get("q");
if ($q) {
    $rows = DB::select("select * from ivr_agent_desks where name like
'%" . $q . "%' and tenant_id = " . $this->tenantId);
}
```

Example 2 — `app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php:28` :

```
$q = $request->get("q");
if ($q) {
    $rows = DB::select("select * from ivr_live_monitorings where name like
'%" . $q . "%' and tenant_id = " . $this->tenantId);
}
```

Example 3 — `app/Http/Controllers/Ivr/CallAnalyticsStoreController.php:28` :

```
$q = $request->get("q");
if ($q) {
    $rows = DB::select("select * from ivr_call_analyticss where name like
'%" . $q . "%' and tenant_id = ".$this->tenantId);
}
```

This pattern repeats identically across all 80 IVR controllers. An attacker sends `?q=' UNION SELECT password FROM users --` and the unescaped value is embedded verbatim in the SQL string, allowing full database extraction via UNION-based injection.

Recommended fix:

1. Replace all raw SQL with parameterized queries: `DB::select("select * from ivr_agent_desks where name like ? and tenant_id = ?", ['%' . $q . '%', $this->tenantId])`
2. Alternatively, migrate to Eloquent query builder: `AgentDesk::where('name', 'like', "%{$q}%")->where('tenant_id', $this->tenantId)->get()`
3. Add input validation to constrain `q` (e.g., max length, alphanumeric)
4. Audit for second-order injection in the 12 Legacy Repository classes that have the same pattern

80 files affected.

File	Line(s)	Issue	Action
app/Http/Controllers/Ivr/AgentDeskDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/AgentDeskExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/AgentDeskImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/AgentDeskIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/AgentDeskStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/AgentDeskSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/AgentDeskUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/BusinessHoursDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/BusinessHoursExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/BusinessHoursImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder

app/Http/Controllers/Ivr/BusinessHoursIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/BusinessHoursStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/BusinessHoursSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/BusinessHoursUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallAnalyticsDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallAnalyticsExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallAnalyticsImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallAnalyticsIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallAnalyticsStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallAnalyticsSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallAnalyticsUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallFlowDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallFlowExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallFlowImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallFlowIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallFlowStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallFlowSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallFlowUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRecordingDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder

app/Http/Controllers/Ivr/CallRecordingExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRecordingImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRecordingIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRecordingStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRecordingSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRecordingUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRoutingDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRoutingExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRoutingImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRoutingIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRoutingStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRoutingSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CallRoutingUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CustomerProfileIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CustomerProfileStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/CustomerProfileUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/DidInventoryDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/DidInventoryExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/DidInventoryImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder

app/Http/Controllers/Ivr/DidInventoryIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/DidInventoryStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/DidInventorySyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/DidInventoryUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/HistoricalReportsExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/HistoricalReportsImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/HistoricalReportsIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/HistoricalReportsStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/HistoricalReportsSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/LiveMonitoringExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/LiveMonitoringImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/LiveMonitoringIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/LiveMonitoringStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/LiveMonitoringSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/PromptLibraryDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder

app/Http/Controllers/Ivr/PromptLibraryExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/PromptLibraryImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/PromptLibraryIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/PromptLibraryStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/PromptLibrarySyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/PromptLibraryUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/QueueManagementDestroyController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/QueueManagementExportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/QueueManagementImportController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/QueueManagementIndexController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/QueueManagementStoreController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/QueueManagementSyncController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder
app/Http/Controllers/Ivr/QueueManagementUpdateController.php	28	SQL injection via string concatenation of request parameter	Replace with parameterized query or Eloquent builder

The same SQL injection pattern also exists in all 12 Legacy Repository files:

Example — `app/Repositories/Legacy/LiveMonitoringRepository.php:14-16` :

```
$sql = "SELECT * FROM ivr_live_monitorings WHERE tenant_id = " . (int)
$tenantId;
if ($filter) {
    $sql .= " AND name LIKE '%" . $filter . "%'";
}
return DB::select($sql);
```

While `$tenantId` is cast to `(int)`, the `$filter` parameter is concatenated without any escaping. Each repository has ~40 methods repeating this pattern (492 total `DB::select` calls).

12 files affected.

File	Line(s)	Issue	Action
app/Repositories/Legacy/AgentDeskRepository.php	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
app/Repositories/Legacy/BusinessHoursRepository.php	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
app/Repositories/Legacy/CallAnalyticsRepository.php	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
app/Repositories/Legacy/CallFlowRepository.php	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
app/Repositories/Legacy/CallRecordingRepository.php	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
app/Repositories/Legacy/CallRoutingRepository.php	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
app/Repositories/Legacy/CustomerProfileRepository.php	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
app/Repositories/Legacy/DidInventoryRepository.php	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
app/Repositories/Legacy/HistoricalReportsRepository.php	14, 23, 32, 41, 50, 59, 68, 77,	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders

	86, 95, 104, 113 +28		
<code>app/Repositories/Legacy/LiveMonitoringRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
<code>app/Repositories/Legacy/PromptLibraryRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders
<code>app/Repositories/Legacy/QueueManagementRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized filter concatenation	Use parameterized queries with bound placeholders

Unsafe `extract()` on Request Data CRITICAL

The `extract()` function is called on unsanitized request payloads in 80 IVR controllers and all 12 Legacy GodService classes (540 total `extract($payload)` calls across services, 4,400 across controllers). `extract()` imports array keys as local variables, allowing an attacker to overwrite any variable in scope — including `$this`, `$service`, `$tenantId`, or any other controller/service variable.

Example 1 — `app/Http/Controllers/Ivr/AgentDeskStoreController.php:52-55` :

```
$payload = $request->all();
extract($payload);
$service = new AgentDeskGodService();
$service->orchestrateAgentDeskWorkflow1($payload);
```

Example 2 — `app/Legacy/Services/QueueManagementGodService.php:15-18` :

```
extract($payload); // unsafe
sleep(1);
self::$sharedRuntimeCache[$tenant_id ?? 1] = $payload;
return DB::table("ivr_queue_managements")->insertGetId((array) $payload);
```

An attacker who sends `{"service": "malicious", "tenantId": 999}` in the request body can overwrite the `$service` and `$tenantId` variables. Combined with the `insertGetId((array) $payload)` call, this allows arbitrary data insertion into any tenant's table.

Recommended fix:

1. Remove all `extract($payload)` calls immediately

2. Access request data explicitly: `$name = $request->input('name')`
3. Use Laravel form request validation to whitelist allowed fields
4. Add `declare(strict_types=1)` to all files

80 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/AgentDeskDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/AgentDeskExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/AgentDeskImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/AgentDeskStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/AgentDeskSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/AgentDeskUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/BusinessHoursDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165,	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

	178, 191 +43		
app/Http/Controllers/Ivr/BusinessHoursExportController.php	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
app/Http/Controllers/Ivr/BusinessHoursImportController.php	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
app/Http/Controllers/Ivr/BusinessHoursIndexController.php	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
app/Http/Controllers/Ivr/BusinessHoursStoreController.php	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
app/Http/Controllers/Ivr/BusinessHoursSyncController.php	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
app/Http/Controllers/Ivr/BusinessHoursUpdateController.php	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
app/Http/Controllers/Ivr/CallAnalyticsDestroyController.php	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
app/Http/Controllers/Ivr/CallAnalyticsExportController.php	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
app/Http/Controllers/Ivr/CallAnalyticsImportController.php	48, 61, 74, 87, 100, 113, 126, 139,	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

	152, 165, 178, 191 +43		
<code>app/Http/Controllers/Ivr/CallAnalyticsIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallAnalyticsStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallAnalyticsSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallAnalyticsUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallFlowDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallFlowExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallFlowImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallFlowIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

<code>app/Http/Controllers/Ivr/CallFlowStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallFlowSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallFlowUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRecordingDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRecordingExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRecordingImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRecordingIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRecordingStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRecordingSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165,	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

	178, 191 +43		
<code>app/Http/Controllers/Ivr/CallRecordingUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRoutingDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRoutingExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRoutingImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRoutingIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRoutingStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRoutingSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CallRoutingUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CustomerProfileIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139,	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

	152, 165, 178, 191 +43		
<code>app/Http/Controllers/Ivr/CustomerProfileStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/CustomerProfileUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/DidInventoryDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/DidInventoryExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/DidInventoryImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/DidInventoryIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/DidInventoryStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/DidInventorySyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

<code>app/Http/Controllers/Ivr/DidInventoryUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/HistoricalReportsExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/HistoricalReportsImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/HistoricalReportsIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/HistoricalReportsStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/HistoricalReportsSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165,	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

	178, 191 +43		
<code>app/Http/Controllers/Ivr/LiveMonitoringExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/LiveMonitoringImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/LiveMonitoringIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/LiveMonitoringStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/LiveMonitoringSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/PromptLibraryDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/PromptLibraryExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/PromptLibraryImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139,	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

	152, 165, 178, 191 +43		
<code>app/Http/Controllers/Ivr/PromptLibraryIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/PromptLibraryStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/PromptLibrarySyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/PromptLibraryUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/QueueManagementDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/QueueManagementExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/QueueManagementImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/QueueManagementIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

<code>app/Http/Controllers/Ivr/QueueManagementStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/QueueManagementSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly
<code>app/Http/Controllers/Ivr/QueueManagementUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Unsafe extract() on unsanitized request data enables variable overwrite	Remove extract() and access request fields explicitly

12 files affected.

File	Line(s)	Issue	Action
<code>app/Legacy/Services/AgentDeskGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/BusinessHoursGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/CallAnalyticsGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/CallFlowGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/CallRecordingGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/CallRoutingGodService.php</code>	15, 23, 31, 39,	Unsafe extract() on unsanitized payload in service	Remove extract() and use explicit parameter access

	47, 55, 63, 71, 79, 87, 95, 103 +33	layer	
<code>app/Legacy/Services/CustomerProfileGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/DidInventoryGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/HistoricalReportsGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/LiveMonitoringGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/PromptLibraryGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access
<code>app/Legacy/Services/QueueManagementGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Unsafe extract() on unsanitized payload in service layer	Remove extract() and use explicit parameter access

Hardcoded API Keys in Source Code HIGH

All 12 Legacy GodService classes contain hardcoded API key strings as private class properties. These keys are committed to the git repository and visible to anyone with read access.

Example 1 — `app/Legacy/Services/QueueManagementGodService.php:11` :

```
private $apiKey = "LEGACY_IVR_KEY_2032"; // hard-coded secret
```

Example 2 — `app/Legacy/Services/CallFlowGodService.php:11` :

```
private $apiKey = "LEGACY_IVR_KEY_2012"; // hard-coded secret
```

Example 3 — `app/Legacy/Services/AgentDeskGodService.php:11` :

```
private $apiKey = "LEGACY_IVR_KEY_2042"; // hard-coded secret
```

All 12 services follow this pattern with unique key values. If these keys grant access to any external service, they are compromised to anyone with repo access and in the git history permanently.

Recommended fix:

1. Move all API keys to environment variables via `.env`
2. Reference via `config()` helper or `env()` in config files
3. Rotate all 12 keys immediately since they are in git history
4. Add secret scanning (e.g., Gitleaks) to CI pipeline

12 files affected.

File	Line(s)	Issue	Action
<code>app/Legacy/Services/AgentDeskGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/BusinessHoursGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/CallAnalyticsGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/CallFlowGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/CallRecordingGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/CallRoutingGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/CustomerProfileGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/DidInventoryGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/HistoricalReportsGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/LiveMonitoringGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/PromptLibraryGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key
<code>app/Legacy/Services/QueueManagementGodService.php</code>	11	Hardcoded API key committed to source	Move to environment variable and rotate key

Missing Authorization / Broken Access Control HIGH

No authorization policies, gates, or middleware exist anywhere in the codebase. The application uses authentication (login required) but has zero authorization checks. Any authenticated user can:

- Create, edit, or delete **any other user** within the same account (including granting themselves **owner** privileges)
- Access all organizations and contacts within the account

- Access all IVR modules without role restrictions

Example 1 — `app/Http/Controllers/UsersController.php:86-102` — any authenticated user can update any other user including setting `owner` flag:

```
$user->update(Request::only('first_name', 'last_name', 'email', 'owner'));
if (Request::get('password')) {
    $user->update(['password' => Request::get('password')]);
}
```

Example 2 — The 80+ IVR legacy API routes in `routes/generated/ivr_legacy_api.php` have **no authentication middleware at all**:

```
Route::prefix("ivr-legacy")->group(function () {
    Route::match(['get', 'post'], 'agent-desk/destroy',
        AgentDeskDestroyController::class);
    // ... 80+ routes, no ->middleware('auth')
});
```

Anyone on the network can hit `/api/ivr-legacy/agent-desk/store` without any authentication.

Recommended fix:

1. Add `->middleware('auth:sanctum')` to the IVR legacy API route group
2. Create Laravel policies for User, Organization, Contact, and IVR models
3. Add `$this->authorize()` checks in all controller actions
4. Restrict `owner` field updates to account owners only

1 file affected.

File	Line(s)	Issue	Action
<code>routes/generated/ivr_legacy_api.php</code>	7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18 +68	API routes lack authentication middleware	Add auth:sanctum middleware to route group

Weak Password Policy HIGH

The user creation and update validation allows nullable passwords with no complexity, length, or strength requirements.

Example — `app/Http/Controllers/UsersController.php:48` :

```
'password' => ['nullable'],
```

This means users can be created with empty passwords or single-character passwords. The same lax rule is applied on update (line 90).

Additionally, the `User` model's `$fillable` array (`app/Models/User.php:23-27`) only lists `name` , `email` , `password` , but the `UserController` writes `first_name` , `last_name` , `owner` , and `photo_path` — the controller bypasses `$fillable` by using `$user->update()` with the request fields directly, which works because Eloquent merges attributes.

Recommended fix:

1. Add password rules: `['nullable', 'min:8', Password::defaults()]`
2. Ensure `$fillable` on User model matches the fields actually written by controllers
3. Consider requiring password on user creation (remove `nullable`)

Hardcoded Demo Credentials in Frontend MEDIUM

The login page ships with pre-filled email and password values in the React form state.

Example — `resources/js/Pages/Auth/Login.tsx:8-10` :

```
const form = useForm({
  email: 'johndoe@example.com',
  password: 'secret',
  remember: false as boolean,
})
```

While this is typical for a demo application, if this code reaches a production deployment, users see pre-filled credentials on the login page that may correspond to a real seeded account. The database seeder (`composer compile` script runs `migrate:fresh --seed`) creates this user.

Recommended fix:

1. Make pre-filled credentials conditional on `APP_ENV=demo` or remove them entirely
2. Ensure the `johndoe@example.com` / `secret` account does not exist in production environments

1 file affected.

File	Line(s)	Issue	Action
<code>resources/js/Pages/Auth/Login.tsx</code>	9	Hardcoded demo credentials shipped in client bundle	Conditionally include demo credentials only in non-production builds

XSS via `dangerouslySetInnerHTML` in Pagination MEDIUM

The shared Pagination component renders pagination link labels using React's `dangerouslySetInnerHTML` , which bypasses React's built-in XSS protection.

Example — `resources/js/Shared/Pagination.tsx:16` :

```
<div
  key={key}
  className="mb-1 mr-1 rounded border px-4 py-3 text-sm leading-4 text-gray-400"
```

```

    dangerouslySetInnerHTML={{ __html: link.label }}
  />

```

And again at line 23:

```

<Link
  key={`link-${key}`}
  className={`...`}
  href={link.url}
  dangerouslySetInnerHTML={{ __html: link.label }}
/>

```

The `link.label` values come from Laravel's paginator, which generates HTML entities like `«` and `»` for the previous/next arrows. While these server-generated labels are typically safe, if any custom paginator or API response injects unsanitized content into the label field, it would execute as HTML in the browser.

Recommended fix:

1. Decode HTML entities server-side and render as text: `{link.label}` instead of `dangerouslySetInnerHTML`
2. If HTML in labels is required, sanitize with a library like DOMPurify before rendering

1 file affected.

File	Line(s)	Issue	Action
<code>resources/js/Shared/Pagination.tsx</code>	16, 23	XSS risk via <code>dangerouslySetInnerHTML</code> rendering pagination labels	Replace with text rendering or sanitize with DOMPurify

Missing Security Headers MEDIUM

No security headers are configured anywhere in the application — no middleware, no `.htaccess`, no web server config. Missing headers include:

- **Content-Security-Policy (CSP)** — no protection against XSS payload injection
- **Strict-Transport-Security (HSTS)** — no enforcement of HTTPS
- **X-Frame-Options** — no clickjacking protection
- **X-Content-Type-Options** — no MIME sniffing protection
- **Referrer-Policy** — no control over referrer leakage

Grep for `Content-Security-Policy`, `X-Frame-Options`, `Strict-Transport`, `X-Content-Type` across `app/`, `config/`, and `public/` returned zero results.

Recommended fix:

1. Add security headers middleware or use a package like `spatie/laravel-csp`
2. Configure headers in web server (Nginx/Apache) or Laravel middleware
3. Start with a restrictive CSP and loosen as needed

Outdated Frontend Dependency: react-router-dom 5.2.0 MEDIUM

The `package.json` pins `react-router-dom` to version `5.2.0`, which is a legacy major version. React Router v5 reached end-of-active-development; the current stable line is v6/v7. While no specific CVE is version-mapped to 5.2.0, using an unmaintained major version means security patches are no longer backported.

Example — `package.json:20` :

```
"react-router-dom": "5.2.0",
```

Additionally, `fakerphp/faker` (`^1.23`) is listed under `require` (production) instead of `require-dev`, which ships test-data generation code to production.

Recommended fix:

1. Migrate `react-router-dom` to v6+ or v7
2. Move `fakerphp/faker` from `require` to `require-dev` in `composer.json`

Image Endpoint Path Traversal Risk MEDIUM

The image endpoint accepts a user-controlled path parameter and forwards all query parameters to the Glide image server.

Example — `app/Http/Controllers/ImagesController.php:12-22` :

```
public function show(Filesystem $filesystem, Request $request, $path)
{
    $server = ServerFactory::create([
        'response' => new SymfonyResponseFactory($request),
        'source' => $filesystem->getDriver(),
        'cache' => $filesystem->getDriver(),
        'cache_path_prefix' => '.glide-cache',
    ]);
    return $server->getImageResponse($path, $request->all());
}
```

The route definition `Route::get('/img/{path}', ...)->where('path', '.*')` accepts any path including directory traversal sequences. While Glide has some built-in path normalization, the combination of `.*` matching and passing `$request->all()` as manipulation parameters could allow reading arbitrary files from the filesystem root if the source driver is misconfigured.

Recommended fix:

1. Add Glide's signature validation to prevent parameter tampering
2. Restrict the `path` parameter to only match expected image paths
3. Validate that the resolved path stays within the intended storage directory

Not observed (frontend checks):

- **FS2 — Secrets / API keys in client code:** No hardcoded tokens, API keys, or secrets found in `resources/js/`. The `VITE_APP_NAME` and `VITE_PUSHER_APP_KEY` env vars are non-sensitive public configuration.
- **FS3 — Auth tokens in browser storage:** No `localStorage` or `sessionStorage` usage found. Authentication uses server-side sessions via Inertia.js (cookie-based).
- **FS5 — Missing frontend security controls (partial):** No `target="_blank"` or `postMessage` usage found. However, CSP is missing (covered under Missing Security Headers above).

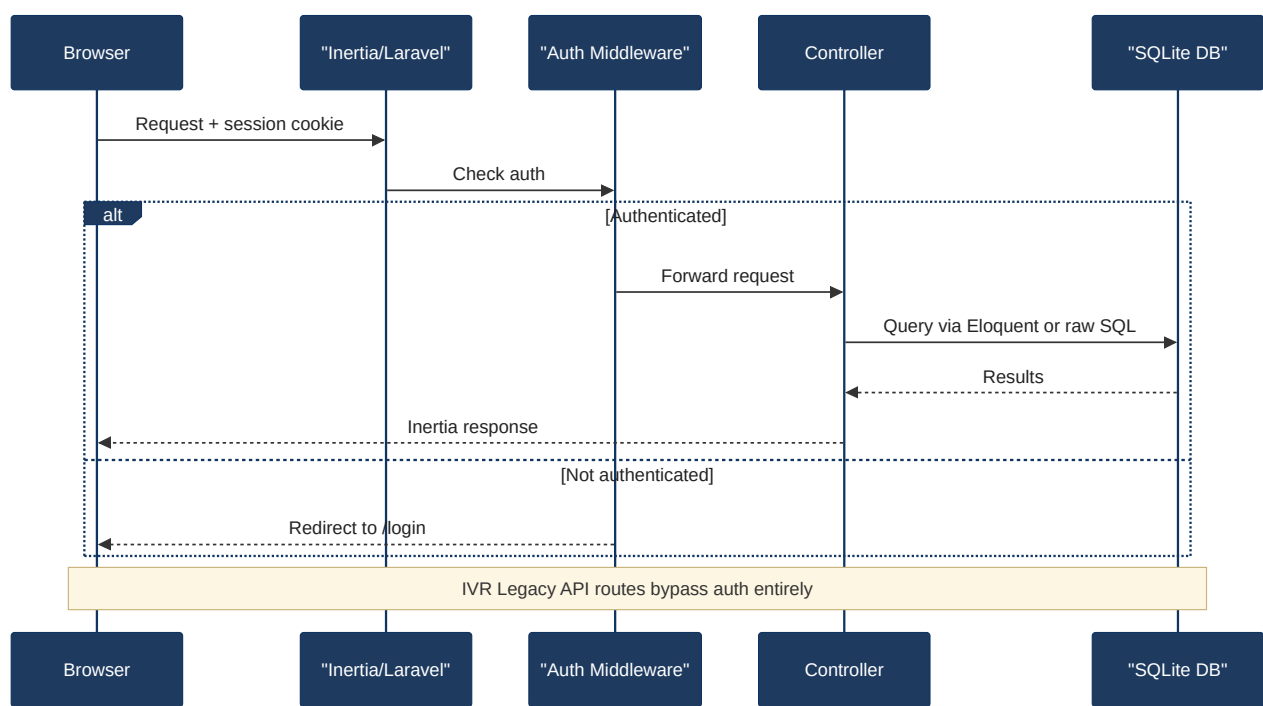
No additional security findings beyond the standard set were observed.

6.3 OWASP Top 10 (2021) Coverage

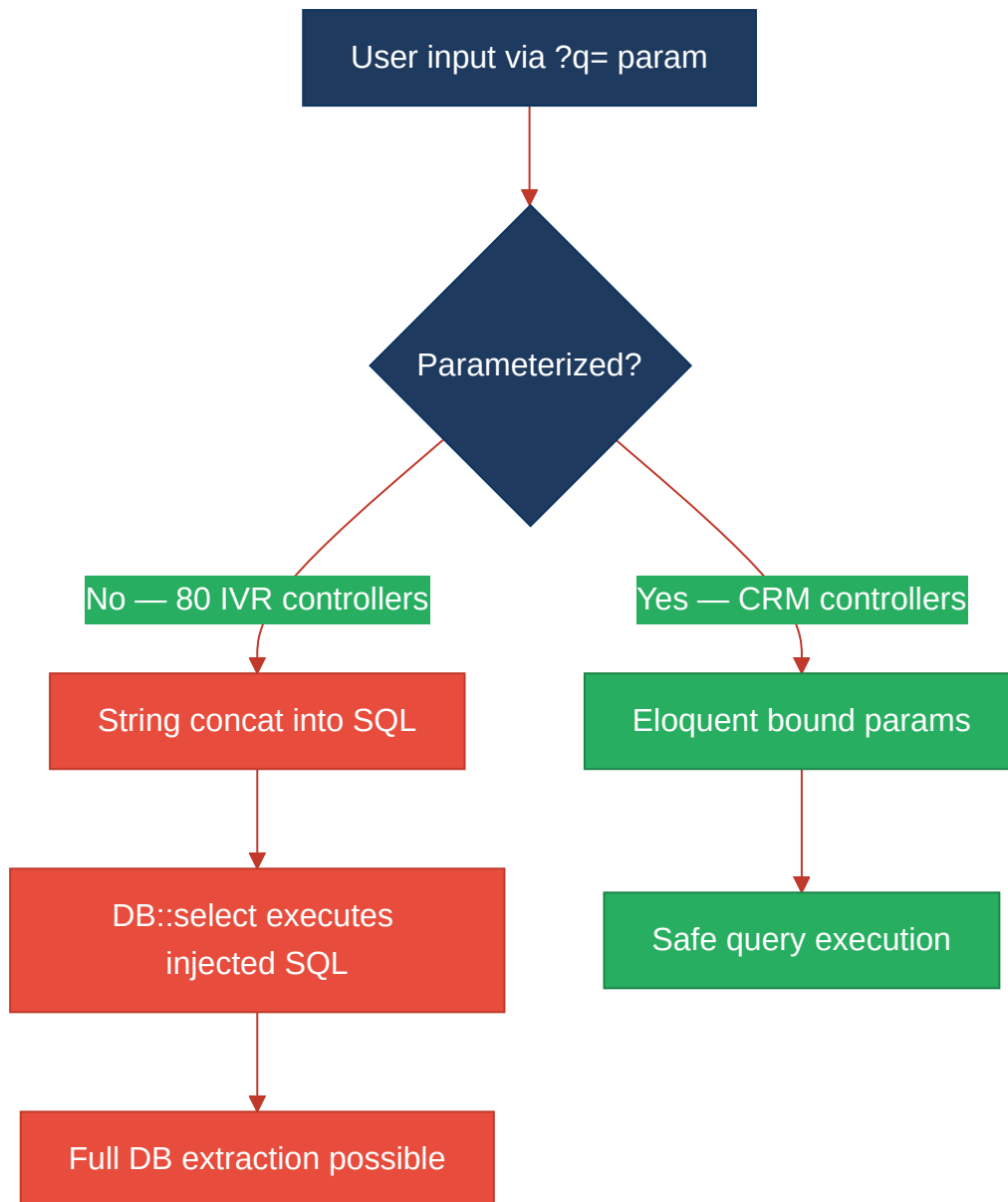
#	Category	Verdict	Evidence / Note
6.1	Broken Access Control	HIGH	No authorization policies/gates; IVR API routes lack auth middleware; any authenticated user can escalate to owner. See §6.2 Missing Authorization.
6.2	Cryptographic Failures	HIGH	12 hardcoded API keys in GodService classes; demo password "secret" in frontend. Password hashing uses Bcrypt via Laravel (good). See §6.2 Hardcoded API Keys.
6.3	Injection	CRITICAL	80 controllers + 12 repositories concatenate user input into raw SQL. 80 controllers + 12 services use <code>extract()</code> on request data. See §6.2 SQL Injection and <code>extract()</code> .
6.4	Insecure Design	MEDIUM	No rate limiting on IVR API routes; hardcoded <code>\$tenantId = 1</code> breaks multi-tenancy; no threat modeling evidence. Login has rate limiting (good).
6.5	Security Misconfiguration	MEDIUM	No security headers (CSP, HSTS, X-Frame-Options); <code>.env.example</code> sets <code>APP_DEBUG=true</code> and <code>SESSION_ENCRYPT=false</code> ; no SAST in CI.
6.6	Vulnerable and Outdated Components	MEDIUM	react-router-dom pinned to EOL v5.2.0; fakerphp/faker in production deps. <code>roave/security-advisories</code> in dev-deps is good.
6.7	Identification and Authentication Failures	MEDIUM	No password complexity rules; passwords can be null. Login rate limiting present (good). Session regeneration on login (good).
6.8	Software and Data Integrity Failures	CLEAN	Composer uses <code>prefer-dist</code> ; <code>composer.lock</code> committed; no insecure deserialization observed.
6.9	Security Logging and Monitoring Failures	MEDIUM	No audit logging for auth events, user modifications, or IVR operations. Default Laravel logging only.
6.10	Server-Side Request Forgery (SSRF)	CLEAN	No server-side HTTP calls built from user-supplied URLs observed. Guzzle is a dependency but not used with user input.
6.11	Other Security Reviews	MEDIUM	Image endpoint (<code>/img/{path}</code>) uses Glide with user-controlled path and passes all query params to image server; potential path traversal depending on Glide config.
6.12	DevSecOps Security Assessment	HIGH	No SAST, no secret scanning, no dependency audit in CI. CI runs tests and static analysis (PHPStan) only. No branch protection evidence.

6.4 Diagrams

Auth / Request Trust Boundary



Top Security Risk Flow — SQL Injection



Improvement Roadmap



6.5 Actions Required

Finding	Action	Rating	Priority
SQL Injection in 80 IVR controllers + 12 repositories	Replace all raw SQL string concatenation with parameterized queries or Eloquent query builder	HIGH RISK	CRITICAL
Unsafe extract() in 80 IVR controllers + 12 GodServices	Remove all extract() calls; access request data explicitly via \$request->input()	HIGH RISK	CRITICAL

Hardcoded API keys in 12 GodService classes	Move keys to environment variables; rotate all keys immediately	HIGH RISK	HIGH
IVR Legacy API routes missing authentication	Add auth:sanctum middleware to ivr-legacy route group	HIGH RISK	HIGH
No authorization policies anywhere	Create Laravel policies for all models; add authorize() checks to controllers	HIGH RISK	HIGH
Weak password validation (nullable, no complexity)	Add min length and complexity rules using Password::defaults()	HIGH RISK	HIGH
No SAST/secret scanning/dependency audit in CI	Add Gitleaks, PHPStan security rules, npm audit, and composer audit to CI	HIGH RISK	HIGH
dangerouslySetInnerHTML in Pagination component	Replace with text rendering or sanitize with DOMPurify	MODERATE	MEDIUM
Missing security headers (CSP, HSTS, X-Frame-Options)	Add security headers middleware or web server config	MODERATE	MEDIUM
Hardcoded demo credentials in Login.tsx	Conditionally include only in demo/local environments	MODERATE	MEDIUM
react-router-dom pinned to EOL v5.2.0	Upgrade to react-router v6+ or v7	MODERATE	MEDIUM
fakerphp/faker in production dependencies	Move from require to require-dev in composer.json	MODERATE	MEDIUM
No audit/security logging	Implement audit logging for auth events, user changes, and IVR operations	MODERATE	MEDIUM
Image endpoint potential path traversal	Add Glide signature validation and restrict path parameter	MODERATE	MEDIUM

6.6 Expected Outcomes

- **Eliminate SQL injection attack surface** by parameterizing all 80+ IVR controllers and 12 repository classes, removing the most critical vulnerability class.
- **Prevent variable overwrite attacks** by eliminating all `extract()` calls on request data across controllers and services.
- **Establish access control** by adding authentication to IVR API routes and authorization policies to all controllers, preventing privilege escalation and unauthorized data access.
- **Automate security assurance** by integrating secret scanning, SAST, and dependency auditing into CI, catching future regressions before they reach production.
- **Harden the frontend** by removing XSS sinks, demo credentials, and adding CSP headers, reducing client-side attack vectors.